



FAST — National University of Computer and Emerging Sciences

Karachi Campus

Department of Artificial Intelligence and Data Science

AL-2002 — Artificial Intelligence Lab

LAB 02

Object-Oriented Programming in Python

Prepared by

Khadeejah Ashraf

Instructor, Department of Artificial Intelligence and Data Science

Fall 2026

Table of Contents

1 Objective.....	4
2 Introduction to Object-Oriented Programming.....	4
2.1 Class.....	4
2.2 Object.....	4
2.3 Attributes and Methods.....	4
2.4 Constructor in Python.....	5
2.5 Types of Attributes in a Class.....	5
2.5.1 Class Attributes.....	6
2.5.2 Instance Attributes.....	6
2.6 Practice Task 1: Smart Light System.....	8
3 Inheritance in Python.....	9
3.1 Basic Concept of Inheritance.....	9
3.2 Inheritance without super().....	9
3.3 Inheritance with super().....	10
3.4 Types of Inheritance.....	10
3.5 Practice Task 2: University Staff System.....	13
4 Polymorphism in Python.....	14
4.1 Compile-Time Polymorphism (Simulated).....	14
4.2 Run-Time Polymorphism.....	15
5 Encapsulation in Python.....	16
5.1 Public Members.....	16
5.2 Protected Members.....	16
5.3 Private Members.....	17
5.4 Practice Task 3: Bank Account System.....	18
6 Useful Standard Modules.....	19
6.1 The math Module.....	19
6.2 The random Module.....	19
7 Lab Tasks.....	21
7.1 Task 1: AI-Powered Security Agents.....	21
7.2 Task 2: Threat Intelligence System.....	21
7.3 Task 3: Automated Threat Response.....	21
7.4 Task 4: Employee Management System.....	21
7.5 Task 5: Student Grading System.....	21
8 Home Tasks and Grading Rubrics.....	23
8.1 Home Task 1: Smart Classroom Devices.....	23

8.2	Home Task 2: Library Management Hierarchy.....	24
8.3	Home Task 3: Debug the Broken Class.....	24
8.4	Home Task 4: Polymorphic Shape Calculator.....	25
8.5	Home Task 5: Encapsulated ATM Simulator.....	26

1 Objective

1. Introduce the concepts of Object-Oriented Programming (OOP) using Python.
2. Develop basic OOP programming skills in Python using classes and objects.
3. Implement the core OOP principles: encapsulation, inheritance and polymorphism.
4. Solve basic programming problems using OOP in Python.

2 Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that organises code using classes and objects to model real-world entities. In Python, OOP allows programmers to create reusable and modular code by encapsulating data and functions inside classes. The key concepts of OOP are encapsulation (hiding internal details), inheritance (reusing code from existing classes), polymorphism (using the same interface for different objects) and abstraction (focusing on the essential features while hiding complexity). Learning OOP in Python helps you design structured programs, improve your problem-solving skills and write software that is easier to maintain and extend.

2.1 Class

A class is a blueprint or template for creating objects. It defines the attributes (data) and the methods (functions) that objects created from it will have. Think of it as a plan for building something — a blueprint for a house, for example.

How to create a class in Python

```
class Student:
    pass
```

2.2 Object

An object is an instance of a class. It represents a real entity created from the class blueprint, with its own values for the attributes.

How to create an object in Python

```
s1 = Student()      # s1 is an object of class Student
s2 = Student()      # s2 is another object
```

2.3 Attributes and Methods

In Python, attributes are variables that belong to a class and hold the data or properties of an object. Methods are functions defined inside a class that describe the behaviours or actions of an object. Attributes store information specific to each object, while methods let objects perform operations using that data. Used together, they organise code in a structured and reusable way.

Declaring an attribute

```
class MyClass:
    x = 5                # x is an attribute
```

Declaring a method

```
class MyClass:
    x = 5                # x is an attribute

    # Method
    def method_one(self):
        pass
```

Accessing attributes and methods

```
print(m1.x)
m1.method_one()
```

Complete example

```
class MyClass:
    x = 5                # x is an attribute

    # Method
    def method_one(self):
        print("This is method one")

m1 = MyClass()          # m1 is an object of MyClass
print(m1.x)
m1.method_one()
```

2.4 Constructor in Python

A constructor is a special method that is called automatically when an object is created. It is used to initialise the attributes of that object. In Python the constructor is always named `__init__()`.

```
class Student:
    def __init__(self, name, age):
        self.name = name    # instance attribute
        self.age = age      # instance attribute

# Creating objects
s1 = Student("Khadeejah", 23)
s2 = Student("Maryam", 22)

print(s1.name, s1.age)     # Output: Khadeejah 23
print(s2.name, s2.age)     # Output: Maryam 22
```

2.5 Types of Attributes in a Class

Attributes are the variables that belong to a class and store data related to objects. There are two main types.

2.5.1 Class Attributes

- Defined directly inside the class, outside any method.
- Shared by every instance of the class.

```
class Student:
    school = "ABC High School"    # class attribute

s1 = Student()
s2 = Student()

print(s1.school)                 # ABC High School
print(s2.school)                 # ABC High School
```

2.5.2 Instance Attributes

- Defined inside the `__init__` constructor using `self`.
- Each object has its own separate copy of these attributes.

```
class Student:
    def __init__(self, name, age):
        self.name = name         # instance attribute
        self.age = age           # instance attribute

s1 = Student("Khadeejah", 23)
s2 = Student("Maryam", 22)

print(s1.name, s1.age)          # Khadeejah 23
print(s2.name, s2.age)          # Maryam 22
```

Modifying class and instance attributes

```
class Student:
    school_name = "ABC School"    # class attribute (shared by all students)

    def __init__(self, name, age):
        self.name = name          # instance attribute
        self.age = age            # instance attribute

    def display_info(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"School: {Student.school_name}")

# Creating student objects (instances)
student1 = Student("Khadeejah", 23)
student2 = Student("Maryam", 22)

student1.display_info()
print("-----")
student2.display_info()

# Modifying an instance attribute - affects only student1
student1.age = 24
print("\nAfter updating the age of student1:")
student1.display_info()

# Modifying the class attribute - affects every object
Student.school_name = "XYZ School"
print("\nAfter updating the school name:")
student1.display_info()
student2.display_info()
```

Worked example: a bank account class

Suppose you are asked to write a program to manage bank accounts. Each account has a holder's name, an account number and a balance, and can display its information or accept a deposit. This scenario shows two benefits of OOP: modularity, because the account logic is separated into its own class, and reusability, because many accounts can be created from that one class without rewriting any code.

```
class BankAccount:
    bank_name = "ABC Bank"          # class attribute shared by all accounts

    def __init__(self, holder_name, account_no, balance):
        self.holder_name = holder_name    # instance attribute
        self.account_no = account_no      # instance attribute
        self.balance = balance            # instance attribute

    def display_account(self):           # method to display account info
        print("Bank:", BankAccount.bank_name)
        print("Account Holder:", self.holder_name)
        print("Account Number:", self.account_no)
        print("Balance:", self.balance)

    def deposit(self, amount):           # method to deposit money
        self.balance += amount
        print(f"Deposited {amount}. New Balance: {self.balance}")

# Create objects / instances
account1 = BankAccount("Khadeejah", 1001, 5000)
account2 = BankAccount("Maryam", 1002, 7000)

# Call methods
account1.display_account()
print("-----")
account2.display_account()
print("-----")
account1.deposit(2000)
```

2.6 Practice Task 1: Smart Light System

Write a program to control the smart lights in a house. Each light has a room name and a status (ON or OFF). The system should allow you to create several light objects for different rooms, turn each light ON or OFF, and display the current status of each light.

Objective: This task demonstrates OOP in AI automation — classes organise devices logically, objects represent the individual lights, and methods automate actions such as switching a light ON or OFF.

Expected output

```
Living Room light turned ON.
Bedroom light turned OFF.
Living Room light is ON.
Bedroom light is OFF.
```


3 Inheritance in Python

Inheritance is the feature of OOP that allows one class (the child or subclass) to reuse the properties and methods of another class (the parent or superclass). It improves code reusability and reduces redundancy.

- **Parent class (superclass):** the class whose properties and methods are inherited.
- **Child class (subclass):** the class that inherits those properties and methods.
- A child class can add new methods of its own or override the ones it inherits.

3.1 Basic Concept of Inheritance

```
class ParentClass:
    # parent class code
    pass

class ChildClass(ParentClass):
    # child class code
    pass
```

3.2 Inheritance without super()

A child object can call an inherited method directly through self. Attributes are different: if the child defines its own `__init__` and never calls the parent's, the parent's attributes are never created.

```
class ParentClass:
    def p(self):
        print('parent func')

class ChildClass(ParentClass):
    def c(self):
        print('child func')
        self.p()                # parent function call

co = ChildClass()
co.c()                         # child func + parent func

# Accessing attributes of the parent class WITHOUT super()
class Parent:
    def __init__(self):
        self.x = 10

class Child(Parent):
    def __init__(self):
        self.y = 20            # Parent.__init__ is never called

c = Child()
print(c.x)                     # AttributeError: 'Child' object has no attribute 'x'
```

3.3 Inheritance with super()

super() gives the child class access to the parent's version of a method, even when the child has overridden it. It is also how a child constructor initialises the attributes defined by the parent.

```
class Parent:
    def p(self):
        print("parent func")

class Child(Parent):
    def p(self):
        print('child func')

    def c(self):
        self.p()           # calls the child version
        super().p()        # calls the parent version

co = Child()
co.c()

# Accessing attributes of the parent class WITH super()
class Parent:
    def __init__(self):
        self.x = 10

class Child(Parent):
    def __init__(self):
        super().__init__() # runs Parent.__init__ and creates self.x
        self.y = 20

c = Child()
print(c.x, c.y)           # 10 20
```

3.4 Types of Inheritance

5. **Single inheritance:** a child class inherits from a single parent class.
6. **Multiple inheritance:** a child class inherits from more than one parent class.
7. **Multilevel inheritance:** a child class inherits from a parent class, which itself inherits from another class.
8. **Hierarchical inheritance:** several child classes inherit from a single parent class.
9. **Hybrid inheritance:** a combination of two or more of the types above.

Single inheritance

```
# Parent class
class Parent:
    def greet(self):
        print("Hello from Parent")

# Child class
class Child(Parent):
    def greet_child(self):
        print("Hello from Child")

c = Child()
c.greet()                # inherited from Parent
c.greet_child()          # child method
```

Multiple inheritance

```
class Parent1:
    def greet_parent1(self):
        print("Hello from Parent1")

class Parent2:
    def greet_parent2(self):
        print("Hello from Parent2")

class Child(Parent1, Parent2):
    def greet_child(self):
        print("Hello from Child")

c = Child()
c.greet_parent1()
c.greet_parent2()
c.greet_child()
```

Multilevel inheritance

```
# Grandparent class
class Grandparent:
    def greet_grandparent(self):
        print("Hello from Grandparent")

# Parent class
class Parent(Grandparent):
    def greet_parent(self):
        print("Hello from Parent")

# Child class
class Child(Parent):
    def greet_child(self):
        print("Hello from Child")

c = Child()
c.greet_grandparent()      # inherited from Grandparent
c.greet_parent()          # inherited from Parent
c.greet_child()           # own method
```

Hierarchical inheritance

```
class Parent:
    def greet_parent(self):
        print("Hello from Parent")

class Child1(Parent):
    def greet_child1(self):
        print("Hello from Child1")

class Child2(Parent):
    def greet_child2(self):
        print("Hello from Child2")

c1 = Child1()
c2 = Child2()
c1.greet_parent()
c1.greet_child1()
c2.greet_parent()
c2.greet_child2()
```

Hybrid inheritance

```
class Grandparent:
    def greet_grandparent(self):
        print("Hello from Grandparent")

class Parent1(Grandparent):
    def greet_parent1(self):
        print("Hello from Parent1")

class Parent2:
    def greet_parent2(self):
        print("Hello from Parent2")

class Child(Parent1, Parent2):
    def greet_child(self):
        print("Hello from Child")

c = Child()
c.greet_grandparent()
c.greet_parent1()
c.greet_parent2()
c.greet_child()
```

3.5 Practice Task 2: University Staff System

You are designing a University Staff Management System that keeps track of teachers, administrative staff and research assistants. All staff share the attributes `name`, `staff_id` and `department`, but each type has its own properties and behaviour: teachers have courses and a salary and can teach courses, admin staff have a role and `working_hours` and perform tasks, and research assistants have a `research_topic` and a stipend and work on research.

Your task: Create a base class `Staff` and derive a class for each staff type, create objects of each and call their methods to simulate a day's activities. Override `display_info()` in every derived class so that it shows the details specific to that type.

4 Polymorphism in Python

Polymorphism means “many forms”. In Python OOP it allows different objects, or different methods with the same name, to respond differently to the same action.

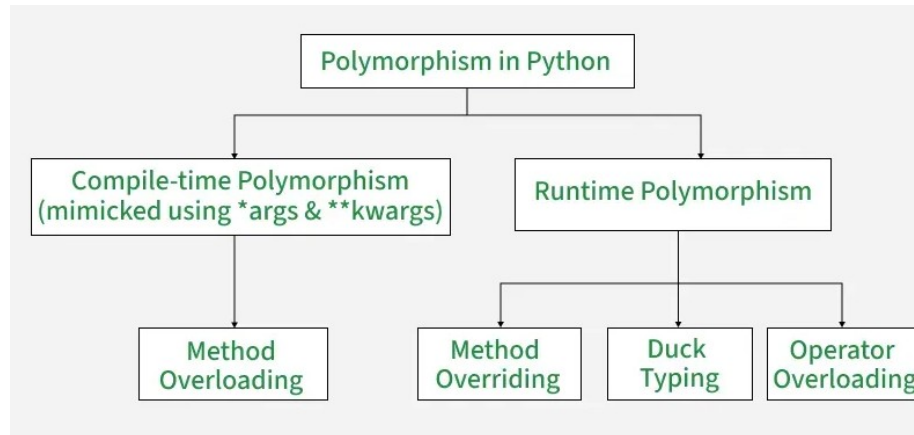


Figure 1: The forms of polymorphism supported in Python.

4.1 Compile-Time Polymorphism (Simulated)

This form of polymorphism is resolved while the program is being compiled. It lets methods or operators with the same name behave differently depending on their parameters. Languages such as Java and C++ achieve it through method overloading, which Python does not support directly.

- True compile-time polymorphism is not available in Python.
- Python imitates it using default arguments or `*args` and `**kwargs`.
- Operator overloading can also be seen as a form of polymorphism, although Python implements it at run time.

4.1.1 Method Overloading using Default Arguments

```
# ----- Method overloading (simulated) using a default argument
class Calculator:
    def add(self, a, b=0):
        """Add two numbers, or one number if only one is given"""
        print("Sum:", a + b)

calc = Calculator()
print("Method Overloading Examples:")
calc.add(5, 10)           # Using two parameters
calc.add(7)              # Using one parameter (default b = 0)
```

4.1.2 Method Overloading using `*args`

```
# ----- Method overloading (simulated) using *args
class Calculator:
    def add(self, *args):
        return sum(args)

calc = Calculator()
print(calc.add(5, 10))          # Two arguments
print(calc.add(5, 10, 15))     # Three arguments
print(calc.add(1, 2, 3, 4))    # Any number of arguments
```

4.2 Run-Time Polymorphism

Run-time polymorphism is resolved while the program is executing. Python provides it in three forms.

10. **Method overriding:** a subclass redefines a method it inherited from its parent class.
11. **Duck typing:** if an object provides the required method it can be used, whatever its type.
12. **Operator overloading:** special methods such as `__add__` and `__sub__` redefine how the operators behave for user-defined objects.

4.2.1 Method Overriding

```
# Parent class
class Animal:
    def sound(self):
        print("Some generic sound")

# Child classes
class Dog(Animal):
    def sound(self):
        print("Bark")          # overrides the parent method

class Cat(Animal):
    def sound(self):
        print("Meow")         # overrides the parent method

# Run-time polymorphism
a = Dog()
b = Cat()
a.sound()                    # Bark
b.sound()                    # Meow

# The same call works on a mixed list - this is the real power of overriding
for animal in [Dog(), Cat(), Animal()]:
    animal.sound()
```

5 Encapsulation in Python

Encapsulation is the OOP principle that restricts direct access to the internal data of a class and bundles the data and the methods that operate on it together. It protects sensitive data, controls how attributes are read or modified, and makes code more secure and maintainable. In Python, encapsulation is expressed through naming: a single leading underscore marks a member as protected, and a double leading underscore marks it as private.

- 13. **Public members:** accessible from anywhere.
- 14. **Protected members:** intended for use inside the class and its subclasses.
- 15. **Private members:** accessible only inside the class that defines them.

5.1 Public Members

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name          # public attribute
        self.roll_no = roll_no    # public attribute

    # public method
    def display_info(self):
        print(f"Name: {self.name}, Roll No: {self.roll_no}")

# Accessing from outside the class
s = Student("Khadeejah", 101)
print("Accessing a public attribute and method from outside the class:")
print(s.name)
print(s.roll_no)
s.display_info()

# Accessing from a subclass
class CollegeStudent(Student):
    def show(self):
        print("\nAccessing a public attribute and method from a subclass:")
        print(self.name)
        print(self.roll_no)
        self.display_info()

cs = CollegeStudent("Maryam", 102)
cs.show()
```

5.2 Protected Members

A single underscore is a convention rather than a restriction: Python still allows access from outside, but the underscore signals that the member is meant for internal use.


```
class Student:
    def __init__(self, name, roll_no):
        self._name = name          # protected attribute
        self._roll_no = roll_no    # protected attribute

    # protected method
    def _display_info(self):
        print(f"Name: {self._name}, Roll No: {self._roll_no}")

s = Student("Khadeejah", 101)
print("\nAccessing a protected attribute and method from outside:")
print(s._name)
print(s._roll_no)
s._display_info()

# Accessing from a subclass - this is the intended use
class CollegeStudent(Student):
    def show(self):
        print("\nAccessing a protected attribute and method from a subclass:")
        print(self._name)
        print(self._roll_no)
        self._display_info()

cs = CollegeStudent("Maryam", 102)
cs.show()
```

5.3 Private Members

A double underscore triggers name mangling, so the attribute cannot be reached by its plain name from outside the class or from a subclass. The correct way to expose it is through a public method.

```
class Student:
    def __init__(self, name, roll_no):
        self.__name = name        # private attribute
        self.__roll_no = roll_no  # private attribute

    # private method
    def __display_info(self):
        print(f"Name: {self.__name}, Roll No: {self.__roll_no}")

    # public method used to reach the private members
    def access_private(self):
        print("Accessing private members inside the same class:")
        print(self.__name)
        print(self.__roll_no)
        self.__display_info()

s = Student("Khadeejah", 101)
s.access_private()

# Accessing from a subclass
class CollegeStudent(Student):
    def show(self):
        print("\nTrying to access private members from a subclass:")
        # The following lines raise AttributeError if uncommented:
        # print(self.__name)
        # self.__display_info()

        # Correct way: use the public method
        self.access_private()

cs = CollegeStudent("Maryam", 102)
cs.show()
```

5.4 Practice Task 3: Bank Account System

You are designing a simple banking system in which every customer has a bank account. The balance must be private, so that it cannot be changed directly from outside the class.

Your task: Create a `BankAccount` class with a private attribute `__balance` and the public methods `deposit()`, `withdraw()` and `get_balance()`. Create a few account objects, perform transactions through those methods, and then try to change `__balance` directly so that you can see how encapsulation protects it.

6 Useful Standard Modules

Python ships with a large standard library. Two modules are used constantly in AI work: `math` for mathematical functions and `random` for generating random values. Both are imported with a single statement and need no installation.

6.1 The `math` Module

```
import math

print("pi:", math.pi)
print("e:", math.e)

# Basic functions
print("ceil(4.2):", math.ceil(4.2))          # nearest upper integer
print("floor(4.8):", math.floor(4.8))        # nearest lower integer
print("fabs(-7.5):", math.fabs(-7.5))        # absolute value as a float
print("factorial(5):", math.factorial(5))    # 5! = 120
print("sqrt(16):", math.sqrt(16))            # square root
print("pow(2, 3):", math.pow(2, 3))          # 2^3 = 8.0
print("exp(2):", math.exp(2))                # e^2
print("log(8, 2):", math.log(8, 2))          # logarithm to base 2
print("log10(100):", math.log10(100))        # logarithm to base 10
print("log2(16):", math.log2(16))            # logarithm to base 2
print("isnan(math.nan):", math.isnan(math.nan)) # check for NaN
print("isinf(math.inf):", math.isinf(math.inf)) # check for infinity

# Trigonometric functions
print("sin(pi/2):", math.sin(math.pi / 2))
print("cos(0):", math.cos(0))
print("tan(pi/4):", math.tan(math.pi / 4))
print("asin(1):", math.asin(1))              # inverse sine
print("acos(1):", math.acos(1))              # inverse cosine
print("atan(1):", math.atan(1))              # inverse tangent
print("atan2(1, 1):", math.atan2(1, 1))      # atan(y / x)
print("degrees(pi):", math.degrees(math.pi))
print("radians(180):", math.radians(180))

# Other useful functions
print("copysign(5, -1):", math.copysign(5, -1)) # 5 with the sign of -1
print("fmod(10, 3):", math.fmod(10, 3))        # remainder, like %
print("gcd(12, 18):", math.gcd(12, 18))        # greatest common divisor
print("lcm(12, 18):", math.lcm(12, 18))        # least common multiple (Python 3.9+)
```

6.2 The `random` Module

```
import random

print("random():", random.random())
print("uniform(1, 10):", random.uniform(1, 10))
print("randint(1, 6):", random.randint(1, 6))
print("randrange(1, 10):", random.randrange(1, 10))
print("randrange(1, 10, 2):", random.randrange(1, 10, 2))

colors = ['red', 'blue', 'green', 'yellow']
print("choice():", random.choice(colors))

nums = [1, 2, 3, 4, 5]
print("choices(k=3):", random.choices(nums, k=3))    # with replacement
print("sample(k=3):", random.sample(nums, 3))       # without replacement

random.shuffle(nums)
print("shuffle():", nums)

random.seed(10)                                     # makes results repeatable
print("seed + randint:", random.randint(1, 100))

print("gauss(0, 1):", random.gauss(0, 1))
print("normalvariate(0, 1):", random.normalvariate(0, 1))
print("expovariate(1.5):", random.expovariate(1.5))
print("triangular(1, 10, 5):", random.triangular(1, 10, 5))
```

7 Lab Tasks

The following five tasks must be completed during the lab session. Each one is built on the concepts covered in this manual: classes and objects, constructors, inheritance, polymorphism and encapsulation. Save each task in a separate file named Lab02_Task<number>.py and show the running program to your instructor.

7.1 Task 1: AI-Powered Security Agents

You are developing a simple AI-powered cybersecurity system that manages different types of security agents. All agents share the attributes `agent_id`, `name` and `status`, but each type has its own specialised role: a `FirewallAgent` monitors network traffic, a `MalwareDetectionAgent` scans files for threats, and an `AutomationAgent` performs routine AI-based tasks to optimise system security.

Your task: Create a base class `SecurityAgent` and derive a class for each agent type, implementing the type-specific methods `monitor_traffic()`, `scan_files()` and `run_automation()`. Create an object of each agent type and call its methods to simulate how a cybersecurity system responds to threats automatically.

7.2 Task 2: Threat Intelligence System

You are building an AI-based threat intelligence system that tracks different types of cybersecurity threats. All threats share the attributes `threat_id`, `name` and `severity`. The derived classes are `PhishingThreat`, which monitors e-mail, `RansomwareThreat`, which monitors the file system, and `BotnetThreat`, which monitors network traffic. Each has its own method: `analyze_email()`, `scan_files()` or `detect_traffic()`.

Your task: Create the base class and the three derived classes, instantiate an object of each, and call their methods to simulate an automated threat response system.

7.3 Task 3: Automated Threat Response

You are designing an AI system that responds to cybersecurity threats automatically. There are three types of response agent: `AlertAgent`, `BlockAgent` and `RecoverAgent`. All of them provide a method `execute_response()`, but each performs a different action — the alert agent sends notifications, the block agent blocks malicious activity, and the recover agent restores the affected systems.

Your task: Create a base class `ResponseAgent` with a generic `execute_response()` method, override it in each derived class, and then demonstrate polymorphism by looping over a list of mixed agent objects and calling `execute_response()` on each of them.

7.4 Task 4: Employee Management System

You are developing a simple employee management system. The base class `Employee` provides a method `work()`. The derived classes `Manager`, `Developer` and `Designer` override `work()` with the tasks specific to their role.

Your task: Create an object of each employee type and call `work()` on each to show how polymorphism allows the same method name to perform different actions.

7.5 Task 5: Student Grading System

You are creating a student grading system. Each student has a private attribute `__grade` holding their score. The `Student` class should provide `set_grade(grade)` to update the score, `get_grade()` to read it, and `display_info()` to show the student's name and grade.

Your task: Create student objects, update their grades through the setter, read them back through the getter, and then attempt to modify `__grade` directly so that you can see how encapsulation keeps it secure.

8 Home Tasks and Grading Rubrics

The following five home tasks are to be completed individually and submitted before the next lab. The five tasks together carry 10 marks: each task is worth 2 marks and is graded against the four-criterion rubric printed beneath it, with each criterion worth 0.5 marks and awarded in full or not at all. Submit one .py file per task, named Lab02_HomeTask<number>_<yourRollNumber>.py, together with a screenshot of the output. Code that does not run will be graded only on the criteria it can satisfy.

Home Task	Title	Marks
1	Smart Classroom Devices	2
2	Library Management Hierarchy	2
3	Debug the Broken Class	2
4	Polymorphic Shape Calculator	2
5	Encapsulated ATM Simulator	2
	Total	10

Table 1: Marks distribution for the home tasks (10 marks in total).

8.1 Home Task 1: Smart Classroom Devices

Problem statement: A smart classroom contains several devices — projectors, air conditioners and smart boards — that a central controller switches on and off. Model the classroom with a single Device class and track how many devices the classroom currently has.

Requirements and constraints:

- The Device class must have a class attribute `total_devices` that counts every object created, and the instance attributes `device_name`, `room` and `status`.
- The constructor must set `status` to 'OFF' by default and increase `total_devices` by one.
- Provide the methods `turn_on()`, `turn_off()` and `display_status()`; switching a device that is already in the requested state must print a message instead of changing anything.
- Create at least four devices across two different rooms, store them in a list, and switch them using a loop.
- Print the value of the class attribute both through the class name and through one object, and explain in a comment why the two are the same.

Expected outcome: A status report for every device before and after the switching loop, ending with a line such as "Total devices in the system: 4".

Criterion	Marks (out of 2)
Class attribute counts objects correctly and instance attributes are set in the constructor	0.5
turn_on(), turn_off() and display_status() behave correctly, including the already-in-state case	0.5
At least four objects created, stored in a list and switched through a loop	0.5
Class attribute accessed both ways with a correct explanatory comment; readable code	0.5
Total	2

8.2 Home Task 2: Library Management Hierarchy

Problem statement: A university library lends printed books, e-books and research journals. Every item shares a title, an item_id and an availability status, but each type is issued under different rules. Model this with an inheritance hierarchy.

Requirements and constraints:

- Write a base class LibraryItem with a constructor and the methods issue_item() and display_info().
- Derive Book (adds author and pages), EBook (adds file_size_mb and download_link) and Journal (adds volume and issue_date).
- Every derived constructor must call the parent constructor with super().__init__() rather than repeating the assignments.
- Every derived class must override display_info() and call super().display_info() inside it, so that the shared details are printed only once.
- Demonstrate multilevel inheritance by deriving ReferenceBook from Book, and add a method that refuses to issue the item.
- Create one object of each class and call display_info() and issue_item() on all of them.

Expected outcome: A printed catalogue in which each item shows its shared details followed by its type-specific details, and in which the reference book refuses to be issued.

Criterion	Marks (out of 2)
Base class and all four derived classes defined with the correct attributes	0.5
Every derived constructor calls super().__init__() with no duplicated assignments	0.5
display_info() overridden in every derived class and extended using super()	0.5
Multilevel inheritance demonstrated by ReferenceBook; all objects created and exercised	0.5
Total	2

8.3 Home Task 3: Debug the Broken Class

Problem statement: The program below is meant to model a student and a college student, but it does not run. Copy it, find every error, correct it, and add a short comment above each correction explaining what was wrong and why your fix works.

```
class Student:
    school = "FAST NUCES"

    def __init__(name, age):
        self.name = name
        self.__age = age

    def display(self):
        print("Name:", name, "Age:", self.__age)

class CollegeStudent(Student):
    def __init__(self, name, age, program):
        self.program = program

    def display(self):
        print("Program:", self.program)
        print("Age:", self.__age)

cs = CollegeStudent("Khadeejah", 23, "AI")
cs.display()
print(cs.school)
```

Requirements and constraints:

- Identify at least four distinct errors and classify each one as a missing self, a scope error, a missing super() call or a private-access error.
- Keep __age private: do not turn it into a public attribute. Expose it through a public getter instead, and explain in a comment why the subclass could not reach it.
- The corrected program must run and print the name, age, programme and school without raising an exception.

Expected outcome: A working program together with a comment block listing every error found, its category and the reasoning behind the fix.

Criterion	Marks (out of 2)
At least four distinct errors identified and correctly classified	0.5
Corrected program runs and prints name, age, programme and school	0.5
__age kept private and exposed through a getter, with the name-mangling issue explained	0.5
super().__init__() used correctly; comments accurate and specific rather than generic	0.5
Total	2

8.4 Home Task 4: Polymorphic Shape Calculator

Problem statement: Build a geometry toolkit that computes the area and the perimeter of different shapes through one common interface, so that new shapes can be added without changing the code that uses them.

Requirements and constraints:

- Write a base class Shape with the methods area() and perimeter() that print a message saying the shape is undefined.
- Derive Rectangle, Circle and Triangle, overriding both methods in each; use the math module for pi and for the square root.
- Simulate method overloading with a method describe(self, unit='cm', *extras) that accepts a default unit and any number of extra notes.
- Store one object of each shape in a list and print the area and perimeter of every element using a single loop — the loop must not test the type of the object.
- Add an unrelated class Square that is not derived from Shape but provides area() and perimeter(); append it to the same list to demonstrate duck typing, and explain the idea in a comment.
- Round every result to two decimal places.

Expected outcome: A table of shapes with their areas and perimeters produced by one loop, including the duck-typed Square, followed by the output of describe() called with and without extra notes.

Criterion	Marks (out of 2)
Base class and three derived shapes implemented with correct area and perimeter formulas	0.5
Both methods overridden in every derived class and results rounded as required	0.5
Single loop over the mixed list with no type checking, demonstrating run-time polymorphism	0.5
Duck typing shown with Square and explained; describe() uses a default argument and *args	0.5
Total	2

8.5 Home Task 5: Encapsulated ATM Simulator

Problem statement: Write an ATM simulator in which the balance and the PIN are fully protected. All access to the account must go through methods that validate the request first.

Requirements and constraints:

- The Account class must keep __balance and __pin private, and _transaction_history protected.
- Provide check_pin(pin), deposit(amount, pin), withdraw(amount, pin) and get_balance(pin); every one of them must reject a wrong PIN before doing anything else.
- A deposit or withdrawal of zero or a negative amount must be refused, and a withdrawal larger than the balance must be refused with a clear message.
- Every successful transaction must be appended to _transaction_history, and a method print_statement(pin) must display the history.

- Demonstrate that direct access such as `account.__balance` fails, and show that the mangled name still exists; explain in a comment why Python calls this convention rather than security.
- Run at least six operations, including two that must be refused.

Expected outcome: A session log showing successful and refused transactions, a final statement listing the history, and the output of the failed direct-access attempt.

Criterion	Marks (out of 2)
<code>__balance</code> and <code>__pin</code> private, <code>_transaction_history</code> protected, all access through methods	0.5
PIN validated in every method before any action is taken	0.5
Invalid amounts and insufficient funds refused with clear messages	0.5
Transaction history recorded and printed; direct-access attempt shown and explained	0.5
Total	2